

Semana Engenharia Agêntica

O aperitivo que vende o Bootcamp — 5 dias de build-along ao vivo, onde o aluno constrói, à mão, um sistema agêntico de analytics do zero à execução em loop.

Caso de uso **TransactCo** — "the analytics are killing the store". Base brownfield real (Postgres → DuckDB → dbt), sistema que já roda, estendido ao vivo por agentes. O aluno vive o **método na mão**; a **automação** que o torna autônomo é o Bootcamp.

FORMATO	DATAS	MECÂNICA	PREÇO	VERSÃO
5 dias · ao vivo	10–14 / 08	~20% teoria ~80% construção	R\$ 97	Ementa v2

PROMPT

CONTEXT

HARNESS

SDD

LOOP

EVAL

Ementa & Plano de Execução

01	Propósito da Semana	o aperitivo estratégico
02	A Fronteira do Moat	o que o aluno faz · o que fica pro Bootcamp
03	O Caso de Uso — TransactCo	brownfield real + o oráculo lacrado
04	A Espinha dos 5 Dias	anel → dia, visão consolidada
05	Dia 1 — Prompt + Context	o documento técnico no Cowork
06	Dia 2 — Harness	na mão → a skill → o wow
07	Dia 3 — Execução	os dois estilos + o cliffhanger · pitch
08	Dia 4 — Loop + Eval	o AHA do oráculo + a régua de Shapiro · pitch
09	Dia 5 — Autoridade Externa	podcast, convidados, CTA
10	O Artefato task-creator	o que mantém, o que corta
11	Semana vs. Bootcamp	por que não canibaliza
12	Riscos & Pré-requisitos	o que travar antes de 10/08

COMO LER ESTE DOCUMENTO

Este é um documento **interno**, de alinhamento entre Luan, Priscila, Cristina e Giulia. Ele nomeia os ativos proprietários (Converge, AgentSpec, o task-loop) para clareza da equipe — mas a seção 02 define exatamente **o que nunca é dito ao aluno ao vivo**. A disciplina de moat vale para o palco, não para esta página.

O propósito da Semana

A Semana não pode ser uma versão pequena do Bootcamp — senão mata a fome em vez de abrir. Ela entrega o resultado; o Bootcamp entrega a engenharia por trás do resultado.

A Semana Engenharia Agêntica é o grande aperitivo do lançamento — o evento ao vivo que aquece a audiência e vende o Bootcamp. Ela se encaixa numa sequência desenhada para levar o público do interesse à compra no pico de intenção:



A sequência do lançamento — cada etapa aquece a próxima; o carrinho fecha na crista da intenção.

O mecanismo de venda é psicológico e comprovado no nosso próprio histórico: **competência gera fome, não saciedade**. O aluno usa peças prontas, sente o poder de construir um sistema que pega erro que humano não pega — mas nunca vê a máquina que torna isso autônomo. Sai sabendo que existe uma engenharia séria que ele ainda não domina. Essa engenharia é o Bootcamp.

O que a Semana entrega

O **resultado** — um sistema agêntico de analytics construído na mão, rodando no laptop do aluno, que investiga dados e pega um erro silencioso pontuado contra a verdade.

O que o Bootcamp entrega

A **engenharia** — como projetar as peças: o harness aterrado, os evals como contrato, o loop que fecha sozinho, a orquestração e a metodologia que compila intenção em sistema.

A PROVA DE QUE FUNCIONA — DO NOSSO PRÓPRIO DADO

Cinco edições anteriores de "Semana" geraram **2.729 vendas** e **~R\$210 mil** de ticket baixo — mas o valor real está no cross-sell: **12,2%** dos compradores migraram para ticket alto (R\$988 mil de receita downstream). A edição mais análoga, a Semana AI Data Engineer 2026, converteu **13,17%** para a Formação. E **66%** dessas conversões acontecem em até 30 dias — exatamente a janela em que o carrinho do Bootcamp fecha (24/08).

A fronteira do moat

O aluno não sai "sem as ferramentas" nem "com o framework". Ele sai com os primitivos despídos — e sentindo, na mão, o trabalho que a automação do Bootcamp elimina.

Esta é a decisão que separa a Semana v2 da v1. A regra é simples e honesta: na Semana, **tudo é feito na mão, com prompt**. Nunca se roda uma skill de automação ao vivo como produto, nunca se diz "Converge", "os 8 passes" ou "o Fork". Os **conceitos** (EDD, unidade atômica, harness, os anéis, os níveis de autonomia) são livres — são vocabulário de indústria. O que se protege é a **encarnação**: as skills que fecham o ciclo, o gate assinado, o loop, o adversário, o Manager que roda a frota. Essa é a diferença entre "eu entendi o conceito" e "eu tenho a máquina".

A SEMANA · O MÉTODO NA MÃO

*O aluno faz tudo com prompt.
Sente a mão. Entrega o resultado.*

- Prompt & Context na mão (o documento técnico)
- Harness montado à mão → depois a skill (o wow)
- task-creator: descreve a tarefa (sem eval, sem loop)
- AgentSpec como caixa-preta (só o resultado)
- Um loop básico fechando UMA tarefa
- Os conceitos: PRD · BDD · SDD · EDD · anéis

O BOOTCAMP · A MÁQUINA

*A automação que torna o método
autônomo. O que não é baixável.*

- O eval como contrato (a definição de "pronto")
- O harness aterrado: KB, doctrine, closers profundos
- O gate assinado + a metodologia de 8 passes
- O adversário cross-model + o Fork
- O Manager / a frota que roda de madrugada
- O flywheel: a fábrica que melhora a fábrica

A fronteira do moat — o método na mão (Semana) de um lado; a automação que o torna autônomo (Bootcamp) do outro.

A ISCA HONESTA — POR QUE DAR TASK-CREATOR E O SCAFFOLDER ABRE O APETITE

Damos dois primitivos de propósito: um **criador de tarefas** (descreve o trabalho, sem eval e sem loop) e um **scaffolder de harness** (gera o esqueleto de agentes e KBs). Com eles a estrutura fica de pé — **mas não roda sozinha**. O que dá autonomia é o método: o eval que define "pronto", o loop que fecha, o adversário que ataca, e o Manager que orquestra a frota de madrugada. O scaffolder cospe *stubs vazios*; o moat é a qualidade do que os preenche. É por isso que ninguém sai da Semana com a fábrica — sai querendo construí-la.

TransactCo — "the analytics are killing the store"

A empresa. A TransactCo é um e-commerce de alto volume. Loja e analytics rodam no mesmo Postgres — `customers`, `products`, `orders`, `payments`. As duas cargas brigam pelos mesmos recursos: os scans analíticos travam o processamento de pedidos, e a carga transacional faz o relatório rastejar. O CFO quer um dashboard de receita confiável; o time sabe que rodar isso no banco de produção é uma bomba-relógio.

A dor real (o gancho). Pior que a lentidão: **os dados mentem em silêncio**. Um `schema_drift` renomeia uma coluna e um pipeline quebra sem erro. Um pagamento duplicado infla a receita. Um pedido órfão referencia um cliente que não existe. Ninguém percebe — até o fechamento do mês não bater.

A MISSÃO DA SEMANA

Em 5 dias, o aluno constrói um sistema agêntico que (1) **entende** o brownfield que já existe, (2) **levanta** o analytics para um DuckDB dedicado, (3) **transforma** em camadas confiáveis, (4) **detecta** os incidentes silenciosos — validado contra um oráculo lacrado — e (5) **fecha o loop** num fluxo que roda com um comando.

O que vem pronto vs. o que o aluno constrói

Vem pronto no repo

A base determinística: o Postgres com schema aplicado no boot, o seeder correlacionado, o gerador de 14 defeitos, o answer key lacrado em `_control.injected_incidents`, e a travessia `make land` (Postgres → DuckDB via ATTACH read-only).

O aluno constrói ao vivo

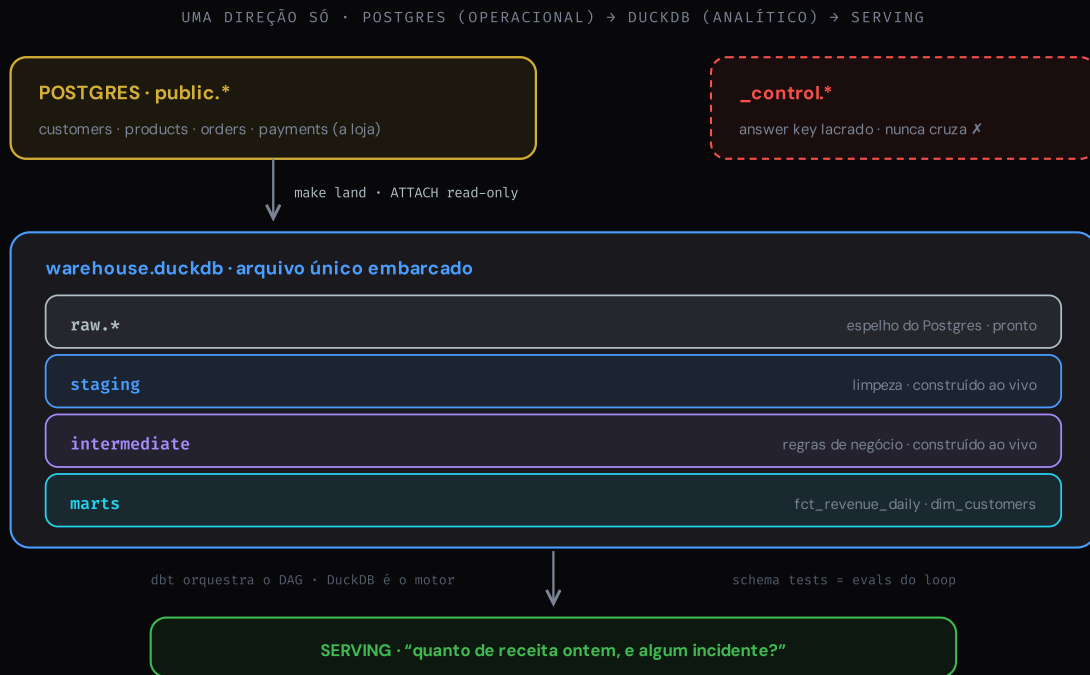
A camada agêntica: os agentes que entendem, movem, transformam e detectam; as transformações dbt em camadas (staging → intermediate → marts); o detector de incidentes; e o loop que amarra tudo num fluxo só.

O ORÁCULO LACRADO — O MOTOR DO AHA

O `_control.injected_incidents` vive num schema separado que **nunca é copiado** para o DuckDB. É a verdade lacrada. No Dia 4, quando o detector do aluno é pontuado contra ele, nasce o momento visceral da Semana — a versão-aperitivo do golden-match do Bootcamp.

Como tudo se encapsula

Da fonte operacional ao serving, o caminho é uma direção só. O que vem pronto é a travessia; o que os agentes constroem ao vivo são as camadas de transformação e a detecção. O warehouse inteiro é um único arquivo DuckDB embarcado — sem servidor, sem container: o aluno fecha o notebook e leva o warehouse num arquivo.



O warehouse inteiro num arquivo único: raw (pronto) → staging → intermediate → marts (ao vivo). O answer key vive fora, lacrado.

Os 14 defeitos injetáveis — a matéria-prima do Dia 4

Qualidade de dado

negative_price · missing_customer ·
invalid_quantity · duplicate_order ·
orphan_payment · malformed_data

Schema & comportamento

schema_drift · late_arrival ·
volume_spike · recurring_incident ·
ambiguous_anomaly · destructive_fix ·
slow_source · multi_failure_cascade

POR QUE É O APERITIVO PERFEITO

Mesmo DNA do caso do Bootcamp (fonte operacional → warehouse → transformação → detecção validada contra a verdade), a um décimo do tamanho: 4 tabelas, 1 costura, 14 defeitos conhecidos, gabarito lacrado — e roda no laptop. É o que torna a Semana um aperitivo, e não o prato principal.

A espinha dos 5 dias

Cada dia acende um anel e remove um pouco mais da mão humana do caminho — sem nunca entregar a máquina que fecha o ciclo sozinha.



Cada dia remove um pouco mais da mão humana do caminho — sem nunca entregar a máquina que fecha o ciclo sozinha.

O loop só aparece no Dia 4 — quando o aluno já confia no que construiu à mão.

DIA	ANEL (O ALUNO VÊ)	SKILL EXPOSTA	O QUE FICA PRO BOOTCAMP	PITCH
D1	Prompt + Context	nenhuma — na mão, no Cowork	Second Brain auto-atualizável (Obsidian + MCP + Schedules)	—
D2	Harness	<code>agents-kbs</code> (só o esqueleto)	KB aterrado, doctrine tunado, closers profundos	—
D3	SDD / execução	<code>task-creator</code> + AgentSpec (caixa-preta)	O eval como contrato, o gate assinado, o nome "Fork"	#1
D4	Loop + Eval	loop básico (1 tarefa)	O Manager / a frota, o adversário, o flywheel	#2
D5	Autoridade externa	nenhuma	—	soft

A REGRA QUE PROTEGE TUDO

Do D1 ao D4, nunca se diz ao vivo: `Converge` · `os 8 passes` · `o Fork` · `safe-to-delegate` · `HMAC`. A linguagem é genérica: "o processo", "spec-driven", "tarefa atômica", "definição de pronto". A revelação da marca é o Bootcamp.

RITMO DE CADA DIA

~20% teoria enquanto mostra na tela · ~80% o aluno construindo junto. Um artefato que roda por dia — encadeados, viram o sistema completo.

O Documento Técnico

O agente entende o sistema que ninguém te explicou — e você escreve o brief que o guia.

Foco: Prompt + Context

Engine: Claude Cowork

Autonomia: HITL total

Entregável: tech-spec

Abertura curta (HTML, ~20 min): o que é Engenharia Agêntica, os anéis, as buzzwords desmistificadas — e então a mão no teclado. O "wow" do D1 não é o slide; é o agente lendo o Postgres da TransactCo e descrevendo as regras implícitas sozinho.

A jornada do dia

- 01 **Apresentar o caso e o repo.** Sobe o brownfield (o Postgres já roda). O aluno vê o sistema que vai estender — não um diretório vazio.
- 02 **Injetar o contexto no Claude Cowork.** Joga todos os documentos do caso num projeto: descrição do negócio, o schema, as dores do CFO. O Cowork vira o repositório de contexto do projeto.
- 03 **Apontar o agente para o sistema.** "Entenda este sistema": ele lê o schema, as FKs, amostra os dados e descreve as regras — pedido nunca antes do cliente, `payments.amount` bate com `orders.total`.
- 04 **Ensinar context selection na prática.** Mostra a diferença entre o agente com o contexto certo (regras corretas) vs. contexto pobre (alucinação plausível). Contexto é curadoria, não despejo.
- 05 **Escrever o documento técnico à mão.** O material vira um brief que responde o problema do CFO — escopo, dados, métrica de sucesso. Feito por prompt no Cowork, sem rodar skill nenhuma.

ENTREGÁVEL DO DIA

- ✓ Claude Cowork com todo o contexto do caso carregado
- ✓ O agente navegando o brownfield e explicando a arquitetura
- ✓ Um documento técnico (o brief) escrito à mão, respondendo o problema

GATE · o sistema entende o legado melhor do que você entendia no começo do dia, e o brief prova.

O GANCHO DE FOME → BOOTCAMP

O aluno montou o contexto *na mão*, documento por documento. No Bootcamp, o mesmo contexto vira um **Second Brain que se alimenta sozinho** — Cowork + Obsidian + MCP + Schedules — um organismo vivo que se mantém atualizado sem você. Aqui ele sente o trabalho; lá ele automatiza.

O Harness

Você para de escrever glue code — e sente, na mão, o que uma skill faz em segundos.

Foco: Harness

Engines: Claude · Codex · Kimi

Autonomia: HITL forte

Entregável: sketch plans + harness

O padrão pedagógico do dia é **faz na mão → traz a skill → o wow**: o aluno monta um pedaço do harness manualmente e sente o trabalho; depois o scaffolder faz o mesmo em segundos. O contraste é a venda.

A jornada do dia

- 01 Entender os problemas e revisar o brief.** A partir do documento do D1, o aluno lista o que precisa ser construído — e onde estão as juntas naturais do sistema.
- 02 Cortar os sketch plans.** Na mão, por prompt: duas raias — transformar (medallion sobre `raw.*`) e servir (API/MCP sobre o gold). Cada plano *enumera* o trabalho e a ordem — as tarefas atômicas são o D3.
- 03 O que é harness, de verdade.** Todo engine tem o mesmo loop genérico (pensa → chama tool → observa → repete); o que **você** constrói em volta — contexto, tools, política, delegação — é o moat. "O modelo é alugado; o harness é construído."
- 04 Montar um par de agentes à mão.** Escreve o `AGENTS.md` / `CLAUDE.md` na frente da turma: um architect (julgamento, sem Bash) + um developer (código, com Bash). Trabalhoso de propósito.
- 05 Trazer a skill — o wow.** Roda o scaffolder: em segundos nasce o esqueleto de agentes + KBs + rules para o stack (dbt, duckdb, fastapi, mcp). A turma sente a diferença entre montar à mão e gerar.
- 06 Multi-model na prática.** Claude orquestra e julga; Codex revisa e ataca; Kimi executa barato em escala. Os três no mesmo repo — sem ainda amarrar as peças num fluxo só.

ENTREGÁVEL DO DIA

- ✓ Dois sketch plans (transformar / servir) enumerando o trabalho
- ✓ Um par de agentes escrito à mão (o "antes")
- ✓ O harness scaffoldado pela skill (o "depois" — o wow)
- ✓ O entendimento de que o harness dá autonomia segura

GATE · o harness está de pé e aterrado; os planos enumeram o que falta construir.

O GANCHO DE FOME → BOOTCAMP

O scaffolder cospe *stubs vazios* — e o aluno vê isso. O que faz um agente commodity vencer é a **qualidade do KB aterrado**, o `doctrine.yaml` tunado e os closers profundos, construídos ao longo de milhares de horas. Isso é exatamente o que se cava no Bootcamp.

Os Dois Estilos

Agora a gente executa — e você aprende que "pronto" precisa de uma definição.

Foco: SDD / execução

Skills: task-creator · AgentSpec

Autonomia: HITL saindo

Pitch: #1

Enquadramento curto (~15 min, cada sigla ancorada num artefato na tela, não em slide): PRD, BDD, SDD, TDD, EDD; as formas de criar (vibe coding, agentic development, spec-driven, tarefas atômicas). Depois, mão na massa: construir duas features, uma com cada estilo, para o aluno ver a **parte da execução** — e só ela.

A jornada do dia

- 01 **As formas de executar.** Do vibe coding ao agentic development ao spec-driven — quando cada uma serve. O campo inteiro convergiu para a mesma ideia: a unidade de trabalho é a spec, não o arquivo.
- 02 **Ler o plano e escolher o estilo.** A partir dos sketch plans do D2, decide o que vira spec acoplado e o que vira tarefas atômicas.
- 03 **Feature A com AgentSpec (caixa-preta).** Constrói a parte de Analytics/dbt no estilo spec-driven — vê o estilo funcionando, sem abrir o plugin por dentro.
- 04 **Feature B com task-creator.** Corta a parte de serve/api em tarefas descritas. Conecta ao harness e vê as tarefas sendo criadas — descrição pura, sem eval e sem loop.
- 05 **O contraste é a aula.** Dois estilos, duas fronteiras de confiança, o mesmo sistema. O aluno aprende *a execução* — não o todo.

OS DOIS ESTILOS DE EXECUÇÃO — O FORK TORNADO CONCRETO (SEM NUNCA DIZER "FORK")

O aluno vê que existem **duas formas** de descer um plano até execução, e constrói uma com cada uma:

Estilo A — spec acoplado (AgentSpec, caixa-preta). Uma feature contida de Analytics/dbt, construída no estilo spec-driven. Mostra-se o *resultado* e o estilo — nunca o passeio pelos comandos e agentes internos do plugin.

Estilo B — tarefas atômicas (task-creator). A parte de serve/api, cortada em unidades. Aqui entra o **task-creator**: ele **descreve** a tarefa e conecta ao harness para vê-las sendo criadas — **sem eval, sem loop**. Só a descrição.

O cliffhanger que arma o Dia 4

O dia termina com duas features construídas — e uma pergunta deliberadamente sem resposta: como é que a gente sabe que isso está pronto?

ENTREGÁVEL DO DIA

- ✓ Uma feature de Analytics construída no estilo spec acoplado
- ✓ Uma feature de serve/api cortada em tarefas atômicas descritas
- ✓ As tarefas conectadas ao harness, sendo criadas ao vivo
- ✓ Clareza sobre quando usar cada estilo de execução

GATE • as duas features existem e o aluno executou com os dois estilos — mas repare que as tarefas ainda não dizem o que é "pronto".

O GANCHO DE FOME → BOOTCAMP

As tarefas do `task-creator` **não têm definição de pronto**. Amanhã a gente descobre por que isso é perigoso — e o que transforma uma descrição num contrato que roda sozinho. **[cliffhanger obrigatório: o pitch #1 abre aqui.]**

A MECÂNICA DO PITCH #1 (FIM DO D3)

O pitch abre **depois** do cliffhanger, com a energia da construção ainda quente. A oferta é o Bootcamp a **R\$997 ao vivo** (de R\$1.997), time-bound. O argumento não é "aprenda mais" — é a lacuna que o próprio aluno acabou de sentir: ele descreveu o trabalho, mas não sabe fechar o ciclo. O timing segue o aprendizado da Semana AIDE: pitch depois do momento "WOW", antes da energia dissipar.

POR QUE O PITCH É NO D3 E NO D4 (E NÃO NO D5)

Histórico: a conversão é front-loaded e a janela quente é curta — **66%** das conversões Semana→ticket alto acontecem em até 30 dias, com forte concentração na própria semana do evento. Dois tiros de oferta nos dias de pico de construção (D3 e D4) capturam a intenção no auge; o D5 fica com o CTA suave e a cauda até o carrinho fechar (24/08).

O Loop Fecha

O que faltava: o eval que define "pronto", e o loop que corre até o verde.

Foco: Loop + Eval

Autonomia: Loop (1 tarefa)

Momento: o AHA do oráculo

Pitch: #2

Ontem as tarefas não tinham definição de pronto. Hoje isso ganha nome: **o eval é o contrato**. Mostramos as técnicas de loop engineering, fechamos um loop básico sobre *uma* tarefa — e deixamos a frota autônoma como o horizonte honesto do Bootcamp, não como demo encenada.

A jornada do dia

- 01 **O que é o Loop.** O primitivo: ler tarefa → agir → rodar o eval → RED alimenta a falha de volta → repete até GREEN. Convergir é o eval passar, não "parecer pronto".
- 02 **Eval Engineering.** Por que "o total fecha" não significa "está certo". O eval é o instinto que separa um sistema que você confia de um que só soa bem.
- 03 **Fechar um loop básico sobre uma tarefa.** Uma tarefa do D3 ganha um eval em bash. RED → revisa → GREEN → abre o PR. Zero edição humana no meio do loop. Real, sem skills de learning, sem flywheel.
- 04 **O AHA do oráculo lacrado.** O eval do loop é o próprio oráculo: o detector dá RED (perdeu o defeito) ou GREEN (pegou, pontuado contra a verdade). O loop simples e o drama do oráculo, num movimento só.
- 05 **A régua de Shapiro + o sneak peek honesto.** Onde o mercado empaca, para onde a fábrica leva. Mostra-se o loop fechando *uma* tarefa de verdade — e descreve-se a frota de madrugada como a visão do Bootcamp.

🔥 O MOMENTO AHA — O EVAL CAÇA O ERRO QUE NINGUÉM DECLAROU

Roda-se `make inject` **em silêncio** — um defeito entra nos dados sem aviso, como um incidente real chegaria. O aluno percebe que os números do mart estão estranhos e constrói um detector. No reveal, abre-se o `_control.injected_incidents` — o answer key lacrado o tempo todo — e o detector é pontuado contra a verdade: pegou o defeito certo, nas linhas certas? Ou só soou plausível? **É o momento que a turma nunca esquece.**

Da mão à fábrica

A RÉGUA DE SHAPIRO — ONDE VOCÊ ESTÁ, PARA ONDE A FÁBRICA LEVA

Os 5 níveis de autonomia (do autocomplete apimentado à Dark Factory) situam o aluno: a maioria do mercado empaca no Nível 2 e acha que terminou. A Semana mostra o Nível 3–4 (o loop assumindo); a Dark Factory (Nível 5) é o horizonte que o **Bootcamp** mira.

ENTREGÁVEL DO DIA

- ✓ Um loop básico fechando UMA tarefa com eval verde (RED→GREEN→PR)
- ✓ O detector pontuado contra o oráculo lacrado — o AHA
- ✓ A régua de Shapiro situando o aluno (Nível 2 → 3–4)
- ✓ O entendimento de que sem eval, dado errado passa como certo

GATE • o loop fecha uma tarefa sozinho e o eval prova — e o aluno sentiu, na pele, por que Eval Engineering não é opcional.

O GANCHO DE FOME → BOOTCAMP

O aluno fechou *um* loop, à mão. A **frota que roda de madrugada** — o Manager que despacha issues em paralelo, observa PRs, resolve dependências e converge para fleet-green — não é baixável: é construída. É a promessa do Bootcamp, e o pitch #2 abre exatamente aqui.

A MECÂNICA DO PITCH #2 (FIM DO D4)

O segundo tiro vem no auge emocional da Semana — logo depois do reveal do oráculo. O argumento muda de tom: não é mais "existe uma lacuna" (D3), é "**você acabou de provar que consegue — agora imagine a frota**". Mesma oferta time-bound (R\$997 ao vivo), com a urgência do carrinho: fecha 24/08. Quem não converteu no D3 converte aqui, com a prova de capacidade na mão.

A HONESTIDADE QUE SEPARA DO HYPE

Nunca prometer a "fábrica rodando sozinha" como coisa pronta ao vivo. O que se demonstra é real: *um* loop fechando *uma* tarefa com eval verde. A frota noturna é descrita como o que ela é — a engenharia que o Bootcamp constrói. Essa honestidade é o que separa autoridade técnica de venda de hype, e o público de engenheiros sente a diferença.

O Podcast

A turma construiu; agora o mercado valida — e o carrinho continua aberto.

Foco: autoridade & social proof

Formato: painel / podcast ao vivo

Pitch: CTA suave

O D5 muda a energia de propósito: depois de quatro dias construindo, um painel com convidados de peso empresta autoridade e preenche o vão entre a Semana (14/08) e o fechamento do carrinho (24/08). Não é venda dura — é validação externa da tese que sustenta o Bootcamp: **o método importa mais que a ferramenta.**

A jornada do dia

- 01 **Recap curto + Q&A.** Abre reconhecendo o que a turma construiu nos 4 dias — mantém quem veio para construir engajado.
- 02 **O painel.** Convidados especiais falam de Agentic Engineering: para onde vai o campo, o que muda no trabalho do engenheiro, onde estão os riscos. Curadoria de temas que reforçam "você precisa do método".
- 03 **Curiosidades e bastidores.** Histórias reais de produção, o que deu errado, o que surpreendeu — o conteúdo que gera conexão e compartilhamento.
- 04 **CTA suave + lembrete de carrinho.** Sem pressão: o carrinho do Bootcamp fecha 24/08, há uma cauda de 10 dias que o D5 alimenta.

DEPENDÊNCIA DURA — TRAVAR CEDO

Convidados para uma data ao vivo (14/08) é risco operacional: disponibilidade, fuso, tech. **Fechar os nomes com antecedência** e ter plano B. O painel precisa ser genuinamente bom — não um infomercial disfarçado.

O GANCHO DE FOME → BOOTCAMP

O D5 não vende duro, mas não é só goodwill: os convidados validando "o método vence a ferramenta" são venda indireta do Bootcamp, no exato momento em que a intenção de compra ainda está quente.

task-creator — a isca, especificada

É o task-spec despido: descreve a tarefa e nada mais. Útil de verdade, seguro para o moat, e desenhado para armar o cliffhanger do Dia 3.

O `task-creator` é o `task-spec` real menos tudo que o torna autônomo. O aluno o usa para sentir como é descrever trabalho para um agente — e sai justamente sem a peça que fecha o ciclo. É o "antes" que dá sentido ao "depois" do Bootcamp.

O QUE MANTÉM (A ISCA)	O QUE CORTA (FICA PRO BOOTCAMP)
Frontmatter — id, título, effort, <code>touches_paths</code> , <code>depends_on</code>	A Zona 2 (evals) — os <code>eval_N()</code> em bash que definem "pronto"
Zona de intenção — goal, contexto, por que a tarefa existe	O gate assinado — <code>safe-to-delegate</code> + o selo HMAC
Guardrails — anti-patterns + do-not-touch (o raio de dano)	O Exit Check — a condição de convergência do loop
O gerador — <code>generate-task-spec.sh</code> cuspiendo um stub descritivo	O conformance — os níveis LO/L1/L2 e o threshold por severidade
Conexão ao harness — ver as tarefas sendo criadas	O task-loop — o runner que executa a tarefa até o verde

POR QUE ISSO É SEGURO E AINDA ASSIM VENDE

Uma tarefa **descrita mas sem eval** é exatamente o "ticket que lê bem e deixa quatro decisões para o chute" — e entrega um número silenciosamente errado. O aluno sente esse buraco no D3, e no D4 descobre que o eval é o que o fecha. A skill completa — com eval, gate assinado e loop — é o coração do Bootcamp.

NOTA DE PRODUÇÃO

O artefato é construível: partir do `task-spec` e remover a Zona 2, o `safe-to-delegate`, o selo HMAC e o loop. Sobra o `generate-task-spec.sh` gerando um stub. Posso montar o pacote quando você quiser.

Por que a Semana não canibaliza o Bootcamp

A diferença é de camada de conhecimento, não de profundidade menor. A Semana entrega o método na mão; o Bootcamp entrega a máquina que o torna autônomo.

EIXO	SEMANA ENGENHARIA AGÊNTICA	BOOTCAMP (NORTHWIND PAY EDP)
O que o aluno faz	executa o método à mão, com prompt	projeta a automação: skills, gates, loop, orquestração
O que ele leva	um sistema agêntico que roda com um comando	a metodologia para construir fábricas de sistemas
Skills	task-creator (sem eval) + scaffolder (esqueleto)	o catálogo completo, com eval, gate assinado e loop
Framework exposto?	não — nome nenhum ao vivo	sim — Converge, AgentSpec, o Fork, os 8 passes
Autonomia	um loop fechando uma tarefa	a frota que roda de madrugada (o Manager)
Emoção que gera	"eu consigo" + fome	"eu domino a engenharia"

O GATILHO DE CONVERSÃO

Competência gera fome, não saciedade. O aluno termina a Semana com prova de capacidade própria e uma lacuna nítida e desejável: ele executou o método usando peças que *alguém* automatizou — e agora quer aprender a construir essa automação. Esse desejo é, literalmente, a proposta de valor do Bootcamp.

A RÉGUA CONTRA A V1

Na v1, o risco era a Semana parecer "o Bootcamp em miniatura". Na v2, a separação é estrutural: **camada de método vs. camada de máquina**. Mesmo que um aluno assista às duas, os conteúdos não se sobrepõem — se completam.

Riscos & pré-requisitos

O que confirmar antes de 10/08 — a Semana só escala se a máquina de aquisição e o onboarding estiverem de pé.

R1 · Disciplina de moat

Do D1 ao D4, ninguém no palco diz "Converge", "os 8 passes", "o Fork" ou "HMAC". Linguagem genérica. Se vazar, o aluno acha que já aprendeu e não compra.

R2 · Demos reais, não slides

A força é ver os agentes executando de verdade. Ensaiar cada dia com o harness real (Claude + Kimi) para garantir que o loop converge ao vivo sem depender de sorte.

R3 · Honestidade no D4

Não prometer a "fábrica autônoma rodando sozinha" como coisa pronta — o Manager/frota ainda é roadmap. Mostrar o loop fechando *uma* tarefa (real) e a frota como visão.

R4 · Cliffhanger do D3

Fechar o D3 explicitando que a tarefa não tem "definição de pronto". Sem esse gancho, parte da turma subestima o task-spec ou acha que já viu o Bootcamp.

R5 · Convidados do D5

Travar nomes cedo (fuso, disponibilidade, tech) e ter plano B. Painel genuinamente bom, com CTA suave e lembrete do carrinho (fecha 24/08).

R6 · Pré-requisito técnico do público

O build-along assume rodar `docker-compose` + `make` no laptop. Se o público for mais leigo, ter um ambiente pré-provisionado (Codespaces) para o D1 não travar na largada.

BLOQUEADORES OPERACIONAIS DO LANÇAMENTO — RESOLVER ANTES

Meta Ads sem sincronizar desde abril (aquisição cega), Clint CRM sem sincronizar desde maio (pipeline cego), e o trigger de onboarding em `public.purchases` confirmado vazio (risco de acesso quebrado para quem comprar). A Semana só escala com a máquina de aquisição e o onboarding de pé.

PARA A PRÓXIMA SESSÃO DE CONSTRUÇÃO

Grade hora a hora de um dia-modelo (sugestão: o D3, o de fronteira de moat mais fina), o pacote do `task-creator`, e a mecânica exata do pitch dos dias 3 e 4 (preço, urgência, gancho).